

[Home](#) » [Databricks](#) » [Databricks Certification](#) »

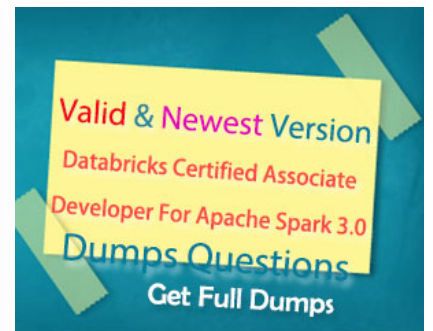
[2022] Real Databricks Certified Associate Developer For Apache Spark 3.0 Exam Dumps | DumpsBase

JANUARY 31, 2022

[2022] Real Databricks Certified Associate Developer For Apache Spark 3.0 Exam Dumps | DumpsBase

Passing Databricks Certified Associate Developer For Apache Spark 3.0 exam to earn the certification can prove that the candidates have demonstrated an understanding of the basics of the Apache Spark architecture and the ability to apply the Spark DataFrame API to complete individual data manipulation tasks. Databricks Certified Associate Developer For Apache Spark 3.0 dumps of DumpsBase containing real exam questions and answers are written by the top experts to ensure that you can pass the actual exam successfully and smoothly. With DumpsBase Databricks Certified Associate Developer For Apache Spark 3.0 Exam Dumps, you will find the Databricks exam extremely easy to reach the top of your IT career via passing the Databricks Certified Associate Developer For Apache Spark 3.0 test by using the real Databricks Certified Associate Developer For Apache Spark 3.0 dumps questions.

Read Databricks Certified Associate Developer For Apache Spark 3.0 Free Dumps First

[Search](#)

Latest Free Dumps

Continue to Read MB-240 Free Dumps (Part 2, Q41-Q80) of V13.02: Verify the Quality of the Microsoft Dynamics 365 Field Service Functional Consultant MB-240 Dumps July 7, 2025

400-007 Dumps (V19.03) Are the Most Current Version with 400 Q&As for Preparation: Check the 400-007 Free Dumps (Part 1, Q1-Q40) to Verify the Quality July 7, 2025

Valid EX370 Dumps for Your Red Hat Certified Specialist in OpenShift Data Foundation Exam Preparation: We have EX370 Free Dumps (Part 2,

Congratulations - you have completed this exam.

Your answers are shown below:

1. Which of the following code blocks silently writes DataFrame itemsDf in avro format to location fileLocation if a file does not yet exist at that location?

- itemsDf.write.avro(fileLocation) ✓
- itemsDf.write.format("avro").mode("ignore").save(fileLocation)
- itemsDf.write.format("avro").mode("errorifexists").save(fileLocati
- itemsDf.save.format("avro").mode("ignore").write(fileLocation)
- spark.DataFrameWriter(itemsDf).format("avro").write(fileLocatio

Question was not answered

Explanation:

The trick in this QUESTION NO: is knowing the "modes" of the DataFrameWriter. Mode ignore will ignore if a file already exists and not replace that file, but also not throw an error. Mode errorifexists will throw an error, and is the default mode of the DataFrameWriter. The QUESTION NO: explicitly calls for the DataFrame to be "silently" written if it does not exist, so you need to specify mode("ignore") here to avoid having Spark communicate any error to you if the file already exists.

The 'overwrite' mode would not be right here, since, although it would be silent, it would overwrite the already-existing file. This is not what the QUESTION NO: asks for.

It is worth noting that the option starting with spark.DataFrameWriter(itemsDf) cannot work, since spark references the SparkSession object, but that object does not provide the DataFrameWriter.

As you can see in the documentation (below), DataFrameWriter is part of PySpark's SQL

API, but not of its SparkSession API.

More info:

DataFrameWriter: pyspark.sql.DataFrameWriter.save — PySpark 3.1.1 documentation

SparkSession API: Spark SQL — PySpark 3.1.1 documentation

Q41-Q80) of V8.02 Online July 7, 2025

PEGACPSA24V1 Free Dumps (Part 2, Q41-Q80): Offering 40 More Demo Questions to Help You Check the PEGACPSA24V1 Dumps (V8.02) July 7, 2025

Share ECBA Free Dumps (Part 2, Q41-Q60) of V10.02 to Verify the Quality: Get Ahead with Entry Certificate in Business Analysis (ECBA) Dumps July 7, 2025

Juniper JN0-253 Free Dumps (Part 3, Q81-Q120) of V8.02 Help You Check the Mist AI, Associate (JNCIA-MistAI) Exam Quality July 7, 2025

Microsoft MS-721 Dumps (V14.02) Accelerate Your Preparation: Continue to Check the MS-721 Free Dumps (Part 2, Q41-Q90) July 5, 2025

CAPM Dumps (V11.02) for Your Certified Associate in Project Management (CAPM) Exam Preparation: Check CAPM Free Dumps (Part 1, Q1-Q40) Today July 5, 2025

Share More Free Demos Online: Read Salesforce Process Automation Free Dumps (Part 2, Q41-Q65) of V13.02 Today July 5, 2025

CompTIA Linux+ XK0-005 Dumps Updated to V16.02 for Quick Preparation: Read XK0-005 Free Dumps (Part 1, Q1-Q40) Online July 5, 2025

Categories

(ISC)² (2)

Static notebook | Dynamic notebook: See test 1,59.(Databricks import instructions)

2. Which of the elements that are labeled with a circle and a number contain an error or are misrepresented?

- 1, 10
- 1, 8 
- 10
- 7, 9, 10
- 1, 4, 6, 9

Question was not answered

Explanation:

1: Correct C This should just read "API" or "DataFrame API". The DataFrame is not part of the SQL API. To make a DataFrame accessible via SQL, you first need to create a DataFrame view. That view can then be accessed via SQL.

4: Although "K_38_INU" looks odd, it is a completely valid name for a DataFrame column.

6: No, StringType is a correct type.

7: Although a StringType may not be the most efficient way to store a phone number, there is nothing fundamentally wrong with using this type here.

8: Correct C TreeType is not a type that Spark supports.

9: No, Spark DataFrames support ArrayType variables. In this case, the variable would represent a sequence of elements with type LongType, which is also a valid type for Spark DataFrames.

10: There is nothing wrong with this row.

More info: Data Types - Spark 3.1.1 Documentation (<https://bit.ly/3aAPKJT>)

3. 5

•

Explanation:

CAP Certification (1)

HCISPP Certification (1)

A10 Networks (1)

System Administration 4 (1)

AACE (4)

AACE Certification (1)

Professional Level
Certifications (3)

AACN (1)

CCRN (1)

AAFM (3)

Chartered Trust and Estate
Planner (1)

Chartered Wealth Manager (2)

AAPC (1)

Certified Professional Coder
(1)

ABA (2)

CRCM Certification (1)

ABPMP (1)

BPM Certification (1)

ACAMS (1)

ACE (1)

Personal Trainer (1)

ACFE (13)

Certified Fraud Examiner (13)

ACI (1)

ACI-Financial Markets
Association (1)

This QUESTION NO: deals with the parameters of Spark's split operator for strings. To solve this question, you first need to understand the difference between `DataFrame.withColumn()` and `DataFrame.withColumnRenamed()`. The correct option here is `DataFrame.withColumn()` since, according to the question, we want to add a column and not rename an existing column. This leaves you with only 3 answers to consider.

The second gap should be filled with the name of the new column to be added to the `DataFrame`. One of the remaining answers states the column name as `itemNameBetweenSeparators`, while the other two state it as `"itemNameBetweenSeparators"`. The correct option here is `"itemNameBetweenSeparators"`, since the other option would let Python try to interpret `itemNameBetweenSeparators` as the name of a variable, which we have not defined. This leaves you with 2 answers to consider.

The decision boils down to how to fill gap 5. Either with 4 or with 5. The QUESTION NO: asks for arrays of maximum four strings. The code in gap 5 relates to the `limit` parameter of Spark's split operator

(see documentation linked below). The documentation states that "the resulting array's

length will not be more than `limit`", meaning that we should pick the answer option with 4 as

the code in the

fifth gap here.

On a side note: One answer option includes a function `str_split`. This function does not exist in `pySpark`.

More info: `pyspark.sql.functions.split` — `PySpark 3.1.2` documentation

Static notebook | Dynamic notebook: See test 3,

4. Which of the following code blocks displays the 10 rows with the smallest values of column value in `DataFrame transactionsDf` in a nicely formatted way?

- `transactionsDf.sort(asc(value)).show(10)`
- `transactionsDf.sort(col("value")).show(10)` ✓
- `transactionsDf.sort(col("value").desc()).head()`
- `transactionsDf.sort(col("value").asc()).print(10)`
- `transactionsDf.orderBy("value").asc().show(10)`

Acquia (1)

Drupal 8 Certification (1)

ACRP (1)

ACRP Certified Professional (1)

ACSM (2)

ACSM-CEP (2)

ACT (2)

ACT English Test (1)

ACT Mathematics Test (1)

Admission Test (2)

Graduate Management Admission Test (1)

Graduate Record Examinations (1)

Adobe (10)

Adobe Advertising (1)

Adobe Campaign Classic (2)

Adobe Commerce (2)

Adobe Experience Manager (1)

Adobe Real-Time CDP (1)

Adobe Workfront (3)

AFP (1)

Certified Treasury Professional (1)

AGA (1)

Government Financial Manager (1)

AHLEI (1)

Question was not answered

Explanation:

show() is the correct method to look for here, since the QUESTION NO: specifically asks for displaying the rows in a nicely formatted way. Here is the output of show (only a few rows shown):

```
+-----+-----+----+-----+-----+----+-----+
|transactionId|predError|value|storeId|productId| f|transactionDate|
+-----+-----+----+-----+-----+----+-----+
| 3| 3| 1| 25| 3|null| 1585824821|
| 5| null| 2| null| 2|null| 1575285427|
| 4| null| 3| 3| 2|null| 1583244275|
+-----+-----+----+-----+-----+----+-----+

```

With regards to the sorting, specifically in ascending order since the smallest values should be shown first, the following expressions are valid:

- transactionsDf.sort(col("value")) ("ascending" is the default sort direction in the sort method)
- transactionsDf.sort(asc(col("value")))
- transactionsDf.sort(asc("value"))
- transactionsDf.sort(transactionsDf.value.asc())
- transactionsDf.sort(transactionsDf.value)

Also, orderBy is just an alias of sort, so all of these expressions work equally well using orderBy.

Static notebook | Dynamic notebook: See test 1,

5. Which of the following code blocks can be used to save DataFrame transactionsDf to memory only, recalculating partitions that do not fit in memory when they are needed?

Certified Hotel Administrator (CHA) (1)

AICPA (3)

AICPA Exam (1)

Public Accountant (1)

AIWMI (1)

Certified Credit Research Analyst (1)

Alcatel Lucent (7)

Alcatel-Lucent Certifications (7)

Alfresco (4)

ACSCA Certification (1)

Alfresco Certification (1)

APSCE Certification (1)

Process Services Administrator (1)

Alibaba Cloud (4)

Alibaba Developer (1)

AliCloud (14)

ACP Certification (1)

Alibaba Cloud Associate (ACA) (8)

Alibaba Cloud Certification Associate (1)

Alibaba Cloud Computing (4)

Amazon (59)

AWS Certification (14)

AWS Certified AI Practitioner (4)

AWS Certified Associate (20)

```

from pyspark import StorageLevel
• transactionsDf.cache(StorageLevel.MEMORY_ONLY)
• transactionsDf.cache()
• transactionsDf.storage_level('MEMORY_ONLY')
• transactionsDf.persist()
• transactionsDf.clear_persist()
  from pyspark import StorageLevel
• transactionsDf.persist(StorageLevel.MEMORY_ONLY)

```



Question was not answered

Explanation:

```

from pyspark import StorageLevel
transactionsDf.persist(StorageLevel.MEMORY_ONLY) Correct.
Note that the storage level MEMORY_ONLY means that all
partitions that do not fit into memory will be recomputed when
they are needed. transactionsDf.cache()

```

This is wrong because the default storage level of
DataFrame.cache() is

MEMORY_AND_DISK, meaning that partitions that do not fit into
memory are stored on disk.

```
transactionsDf.persist()
```

This is wrong because the default storage level of
DataFrame.persist() is

MEMORY_AND_DISK.

```
transactionsDf.clear_persist()
```

Incorrect, since clear_persist() is not a method of DataFrame.

```
transactionsDf.storage_level('MEMORY_ONLY')
```

Wrong. storage_level is not a method of DataFrame.

More info: RDD Programming Guide - Spark 3.0.0
Documentation, pyspark.sql.DataFrame.persist — PySpark 3.0.0
documentation (<https://bit.ly/3sxHLVC> , <https://bit.ly/3j2N6B9>)

6. spark.read.json(filePath, schema=schema)

**C. spark.read.json(filePath,
schema=schema_of_json(json_schema))**

AWS Certified Data Analytics –
Specialty (2)

AWS Certified Database –
Specialty (1)

AWS Certified Developer –
Associate (1)

AWS Certified Professional (5)

AWS Certified Security –
Specialty (1)

AWS Certified Specialty (6)

AWS Certified SysOps
Administrator – Associate (1)

Data Engineer Associate (2)

DevOps-Engineer (1)

American College (1)

American College Certification
(1)

AMPP (1)

Coatings Inspector Program
(1)

Android (4)

Android Certified Application
Developer (1)

Android certified Application
Engineer (2)

Flutter Application
Development (1)

APA (2)

AICP Certification (1)

American Planning Association
(1)

APBM (1)

D. `spark.read.json(filePath, schema=spark.read.json(json_schema))`

•

Explanation:

Spark provides a way to digest JSON-formatted strings as schema. However, it is not trivial to use. Although slightly above exam difficulty, this QUESTION NO: is beneficial to your exam preparation, since

it helps you to familiarize yourself with the concept of enforcing schemas on data you are reading in - a topic within the scope of the exam.

The first answer that jumps out here is the one that uses `spark.read.schema` instead of `spark.read.json`. Looking at the documentation of `spark.read.schema` (linked below), we notice that the operator expects types `pyspark.sql.types.StructType` or `str` as its first argument. While variable `json_schema` is a string, the documentation states that the `str` should be "a DDL-formatted string (For example `col0 INT, col1 DOUBLE`)". Variable `json_schema` does not contain a string in this type of format, so this answer option must be wrong.

With four potentially correct answers to go, we now look at the `schema` parameter of `spark.read.json()` (documentation linked below). Here, too, the `schema` parameter expects an input of type `pyspark.sql.types.StructType` or "a DDL-formatted string (For example `col0 INT, col1 DOUBLE`)". We already know that `json_schema` does not follow this format, so we should focus on how we can transform `json_schema` into `pyspark.sql.types.StructType`. Hereby, we also eliminate the option where `schema=json_schema`.

The option that includes `schema=spark.read.json(json_schema)` is also a wrong pick, since `spark.read.json` returns a `DataFrame`, and not a `pyspark.sql.types.StructType` type.

Ruling out the option which includes `schema_of_json(json_schema)` is rather difficult. The operator's documentation (linked below) states that it "[p]arses a JSON string and infers its schema in DDL format". This use case is slightly different from the case at hand: `json_schema` already is a schema definition, it does not make sense to "infer" a schema from it. In the documentation you can see an example use case which helps you understand the difference better. Here, you pass

CBM Certifications (1)

APEGS (1)

Professional Practice Exam (1)

API (4)

ICP Programs Certification (1)

Individual Certification
Programs (3)

APICS (8)

CLTD Certification (1)

CPIM Certification (3)

CSCP Certification (3)

Supply Chain Professional (1)

APM (4)

APM Project Fundamentals
Qualification (PFQ) (3)

Project professionals (1)

APMG-International (12)

Agile Business Analysis (2)

AgilePM (5)

Better Business Cases (2)

Change Management (3)

Appian (11)

Associate Developer (4)

Certified Analyst (1)

Lead Developer (3)

Senior Developer (3)

Apple (4)

Apple Certified iOS Technician
(ACiT) 2019 (1)

string '{a: 1}' to `schema_of_json()` and the method infers a DDL-format schema `STRUCT<a: BIGINT>` from it.

In our case, we may end up with the output schema of `schema_of_json()` describing the schema of the JSON schema, instead of using the schema itself. This is not the right answer option.

Now you may consider looking at the `StructType.fromJson()` method. It returns a variable of type `StructType` - exactly the type which the schema parameter of `spark.read.json` expects. Although we could have looked at the correct answer option earlier, this explanation is kept as exhaustive as necessary to teach you how to systematically eliminate wrong answer options.

More info:

- `pyspark.sql.DataFrameReader.schema` — PySpark 3.1.2 documentation

- `pyspark.sql.DataFrameReader.json` — PySpark 3.1.2 documentation

- `pyspark.sql.functions.schema_of_json` — PySpark 3.1.2 documentation

Static notebook | Dynamic notebook: See test 3,

7. "left_semi"

•

Explanation:

Correct code block:

```
transactionsDf.join(broadcast(itemsDf), "transactionId",  
"left_semi")
```

This QUESTION NO: is extremely difficult and exceeds the difficulty of questions in the exam by far.

A first indication of what is asked from you here is the remark that "the query should be executed in an optimized way". You also have qualitative information about the size of `itemsDf` and `transactionsDf`. Given that `itemsDf` is "very small" and that the execution should be optimized, you should consider instructing Spark to perform a broadcast join, broadcasting the "very small" DataFrame `itemsDf` to all executors. You can explicitly suggest

Apple Certified IT Professional (ACITP) (1)

Apple Certified Support Professional (2)

APSE (1)

Employment Support Professional (1)

Arcitura (6)

Cloud Certified Professional (CCP) (5)

SOA Architect (1)

ARDMS (1)

ARDMS SPI (1)

Arista (1)

Arista Certification (1)

Aruba (1)

Aruba ACMP (1)

ASCP Cert (1)

MLT(ASCP) (1)

ASIS (4)

Certified Protection Professional (3)

Professional Certified Investigator (1)

ASQ (9)

ASQ Certification (8)

Quality Improvement Associate (1)

ATD (1)

Talent Development (1)

Atlassian (17)

this to Spark via wrapping itemsDf into a broadcast() operator. One answer option does not include this operator, so you can disregard it. Another answer option wraps the broadcast() operator around transactionsDf - the bigger of the two DataFrames. This answer option does not make sense in the optimization context and can likewise be disregarded.

When thinking about the broadcast() operator, you may also remember that it is a method of pyspark.sql.functions. One answer option, however, resolves to itemsDf.broadcast([...]). The DataFrame

class has no broadcast() method, so this answer option can be eliminated as well.

All two remaining answer options resolve to transactionsDf.join([...]) in the first 2 gaps, so you will have to figure out the details of the join now. You can pick between an outer and a left semi join. An outer join would include columns from both DataFrames, where a left semi join only includes columns from the "left" table, here transactionsDf, just as asked for by the question. So, the correct answer is the one that uses the left_semi join.

8. parquet

•

Explanation:

Correct code block:

```
transactionsDf.write.format("parquet").mode("overwrite").option("compression",  
"snappy").save(storeDir)
```

Solving this QUESTION NO: requires you to know how to access the DataFrameWriter (link below) from the DataFrame API - through DataFrame.write.

Another nuance here is about knowing the different modes available for writing parquet files that determine Spark's behavior when dealing with existing files. These, together with the compression

options are explained in the DataFrameWriter.parquet documentation linked below.

ACP-OA (2)

Jira Administrator (10)

Jira Project Administrator (4)

Jira service management (1)

Auldhouse (1)

MSPF Certification (1)

Autodesk (1)

Autodesk Maya (1)

Automation Anywhere (1)

RPA Certifications (1)

Avaya (95)

ACDS (7)

ACIS (27)

ACSS (38)

Administering Avaya Call
Management System (ASAC-
0021) (1)

APDS (3)

APSS (7)

Avaya Aura Communication
Applications Technical
Associate Support (ASTA-
7230) (1)

Avaya Aura Communication
Manager Administrator (ASAC-
0038) (1)

Avaya Experience Portal with
Proactive Outreach Manager
Support (ACSS-3305) (1)

Avaya Experience Portal with
Proactive Outreach Manager
Technical Associate Support
(ASTA-3305) (1)

Finally, bracket __5__ poses a certain challenge. You need to know which command you

can use to pass down the file path to the DataFrameWriter. Both save and parquet are

valid options here.

More info:

- DataFrame.write: pyspark.sql.DataFrame.write — PySpark 3.1.1 documentation

- DataFrameWriter.parquet:
pyspark.sql.DataFrameWriter.parquet — PySpark 3.1.1 documentation

Static notebook | Dynamic notebook: See test 1,

**Pass Your Databricks
Certified Professional Data
Engineer Exam with
Confidence: Practice with
222 Dumps Questions**

**Real Databricks Certified
Professional Data Scientist
Dumps With Actual
Questions**

Tags: [Databricks Certified Associate Developer For Apache Spark 3.0](#), [Databricks Certified Associate Developer For Apache Spark 3.0 dumps](#), [Databricks Certified Associate Developer For Apache Spark 3.0 dumps questions](#), [Databricks Certified Associate Developer For Apache Spark 3.0 exam dumps](#), [Databricks Certified Associate Developer For Apache Spark 3.0 free dumps](#)

Related Posts

What is the Difference between the Databricks Certified Professional Data Engineer and Databricks

Databricks Certified Data Analyst Associate Exam Dumps Online: Get Benefits from DumpsBase

Get the Latest Databricks Certified Data Engineer Associate Dumps (V10.02) to

Avaya IP Office™ Platform Administrator (ASAC-0013) (1)

Avaya IP Office™ Platform Technical Associate Implement (ASTA-7720) (1)

Avaya IP Office™ Platform Technical Associate Support (ASTA-3000) (2)

Avaya Solutions Administrator Certificate (1)

Avaya Workforce Engagement Technical Associate Support (ASTA-3308) (1)

Avaya-Certification (1)

AXP On-Prem (formerly Avaya Aura CC Elite) Support (ACSS-7492) (1)

AXP On-Prem (formerly Avaya Aura CC Elite) Technical Associate Implement (ASTA-7392) (1)

AXP On-Prem Technical Associate Support (ASTA – 7492) (1)

Aviatrix (2)

ACE Certifications (2)

AVIXA (1)

Certified Technology Specialist (CTS) (1)

Axis (2)

Axis Certified Professional (2)

BACB (2)

BACB certification (2)

BCM Institute (1)

BCMS Audit Certification (1)